HEALTHCARE ANALYTICS & HOSPITAL MANAGEMENT SQL SYSTEM

NAME : Janhavi Kotkar
BATCH : KL352
TRAINER : Zakaur Khan Sir

PROJECT OBJECTIVE

- **Relational Schema Design :** Designed a multi-table database schema (Patients, Doctors, Appointments, Billing) using DDL statements.
- **Data Integrity & Normalization :** Enforced Primary Keys and Foreign Keys to preserve data accuracy and eliminate redundancy.
- **ETL & Data Transformation :** Processed and aggregated raw clinical data into analytical summary tables using DML routines.
- **Complex Data Integration :** Linked patient histories, doctor consultations, and financial records via multi-table Joins (INNER, LEFT).
- **Advanced SQL Analytics :** Analyzed readmission rates, bed occupancy, and revenue using CTEs and Window Functions (RANK(), SUM() OVER()).
- **Visual Data Modeling :** Generated a complete EER Diagram in MySQL Workbench to document the database architecture.

PROJECT OVERVIEW

Project Name : Healthcare Analytics & Hospital Management SQL System
Total Records : ~1,000+ Records (across all relational entities)

Dataset Schema and Attributes :

Patients :
patient_id
first_ name
last _ name
gender
date of birth
phone number
address

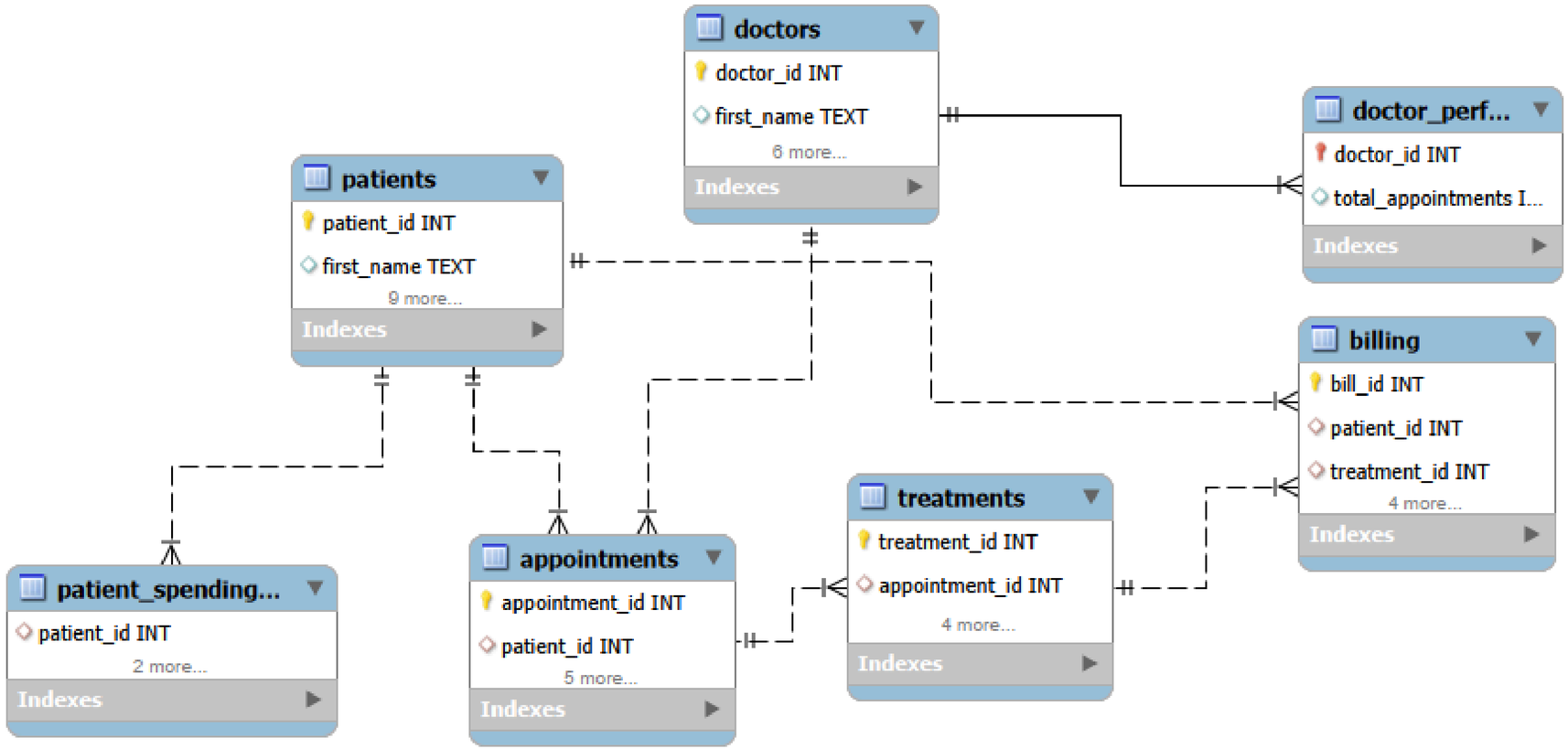
Doctors:
doctor_id
first_name
last_name
specialization
phone number
email
experience_years

Appointments:
appointment_id
patient_id
doctor_id
appointment_date
status

Treatments:
treatment_id
appointment_id
diagnosis
prescribed_medicine
treatment_cost
treatment_date

Billing:
bill_id
patient_id
treatment_id
total_amount
payment_status
payment_date

DATABASE DESIGN : (ER DAIGRAM)



ONE - TO - MANY (1 TO *)
RELATIONSHIP

DDL : Data Definition Language

A subset of SQL used to define, manage, and modify the structure of a database.

CORE COMMANDS :

CREATE: Builds a new database, table, index, or view from scratch.

ALTER: Modifies the existing structure of a database object (e.g., adding or removing a column).

DROP: Completely deletes an object from the database.

TRUNCATE: Empties all data records from a table, but keeps the table structure intact.

RENAME: Changes the name of an existing database object.

QUERIES

RENAME :
Used to change the name of an existing table without affecting its data or underlying structure.

```
-- RENAME TABLE
alter table patient
rename to patients;
```

DROP :
Permanently deletes a table along with all its data, structure, and associated indexes from the database.

```
-- DROP COLUMN
create table patients_backup as
select*from patients;

drop table if exists patients_backup;
```

TRUNCATE :
Removes all records from a table instantly while retaining its original schema for future data entry.

```
-- TRUNCATE
truncate table appointments;
```

DML : Data Manipulation Language

Data Manipulation Language (DML) is a subset of SQL used to insert, modify, retrieve, and delete data within existing database tables without altering the database structure.

DML COMMANDS:

SELECT: Retrieves specific records from one or more tables based on defined conditions.

INSERT: Adds new rows of data into a specified table.

UPDATE: Modifies existing data values in one or more table rows.

DELETE: Removes specific rows of data from a table based on a given condition.

QUERIES

INSERT QUERY:

```
-- INSERT
insert into patients (patient_id,first_name,last_name,gender,date_of_birth,contact_number)
values ("P999","mery","roy","F","1990-05-15","5550199");
```

UPDATE QUERY:

```
-- UPDATE
update patients
set last_name = 'Smith'
where patient_id = 'P999';
```

DELETE QUERY:

```
-- DELETE
delete from patients
where patient_id = 'P999';
```

DQL : Data Query Language

Data Query Language (DQL) is a subset of SQL used to retrieve and search data stored in a database without modifying its content or structure.

DQL COMMANDS :

SELECT : Fetches specific data columns or rows from one or more tables based on defined conditions.

WHERE : Filters records to return only those that meet specific logical criteria.

GROUP BY : Aggregates data into summary rows based on shared values across specified columns.

HAVING : Filters aggregated groups generated by GROUP BY based on specific conditions.

ORDER BY : Sorts the retrieved result set in ascending (ASC) or descending (DESC) order.

SELECT :

Fetches specific data columns or rows from one or more tables based on defined conditions.

For example:

QUERY :

```
select * from treatments;
```

OUTPUT :

Result Grid | Filter Rows: | Export: | Wrap Ce

treatment_id	appointment_id	treatment_type	description
T001	A001	Chemotherapy	Basic screening
T002	A002	MRI	Advanced protocol
T003	A003	MRI	Standard procedure
T004	A004	MRI	Basic screening
T005	A005	ECG	Standard procedure
T006	A006	Chemotherapy	Standard procedure

treatments 1

QUERY :

```
select * from patients;
```

OUTPUT :

Result Grid | Filter Rows: | Export:

patient_id	first_name	last_name	gender	date_of_birth
P001	David	Williams	F	04-06-1955
P002	Emily	Smith	F	12-10-1984
P003	Laura	Jones	M	21-08-1977
P004	Michael	Johnson	F	20-02-1981
P005	David	Wilson	M	23-06-1960
P006	Linda	Jones	M	16-06-1963

patients 2

WHERE CLAUSE :

Filters records to return only those that meet specific logical criteria.

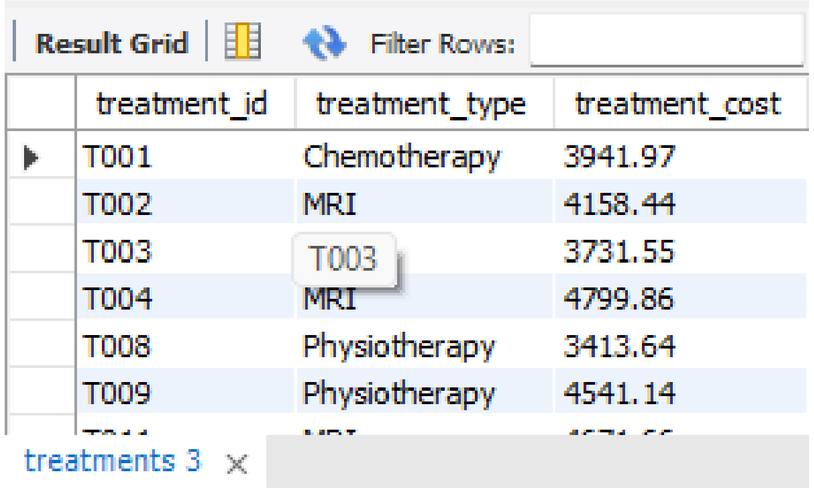
For Example :

find all the treatments where the cost is greater than 3000

QUERY :

```
select
treatment_id,treatment_type,treatment_cost
from treatments
where treatment_cost > 3000;
```

OUTPUT :



Result Grid | Filter Rows:

treatment_id	treatment_type	treatment_cost
T001	Chemotherapy	3941.97
T002	MRI	4158.44
T003		3731.55
T004	MRI	4799.86
T008	Physiotherapy	3413.64
T009	Physiotherapy	4541.14

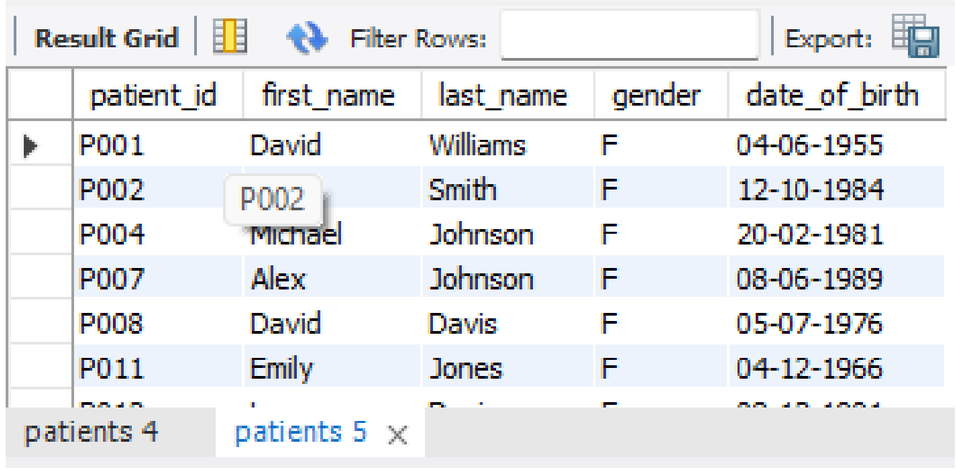
treatments 3

retrieve all female patients from the database

QUERY :

```
select
patient_id, first_name, last_name, gender,
date_of_birth
from patients
where gender="F";
```

OUTPUT :



Result Grid | Filter Rows: | Export:

patient_id	first_name	last_name	gender	date_of_birth
P001	David	Williams	F	04-06-1955
P002		Smith	F	12-10-1984
P004	Michael	Johnson	F	20-02-1981
P007	Alex	Johnson	F	08-06-1989
P008	David	Davis	F	05-07-1976
P011	Emily	Jones	F	04-12-1966

patients 4 | patients 5

GROUP BY CLAUSE :

Aggregates data into summary rows based on shared values across specified columns.

For Example :

what is the total billing amount collected for each payment status

QUERY :

```
select payment_status,
sum(amount) as total_amount,
count(bill_id) as total_bills
from billing
group by payment_status;
```

OUTPUT :

Result Grid | Filter Rows:

payment_status	total_amount	total_bills
Pending	2.01000000004	69
Paid	173424.89999999994	64
Failed	193212.94	67

how many doctors belong to each specialization

QUERY :

```
select specialization,
count(doctor_id) as total_doctors
from doctors
group by specialization;
```

OUTPUT :

Result Grid | Filter Rows:

specialization	total_doctors
Dermatology	3
Pediatrics	5
Oncology	2

HAVING CLAUSE :

Filters aggregated groups generated by GROUP BY based on specific conditions.

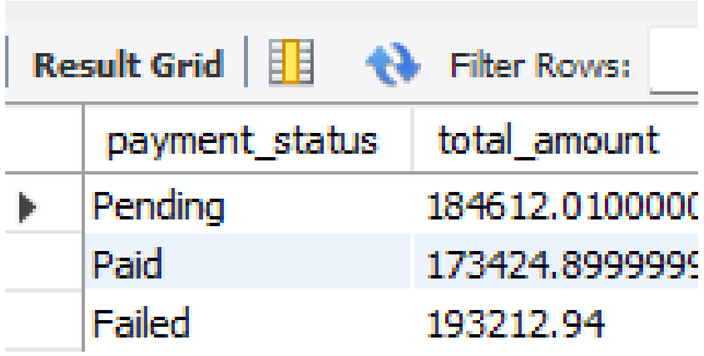
For Example :

which payment statuses have a total billed amount greater than 5000

QUERY :

```
select payment_status,
sum(amount) as total_amount
from billing
group by payment_status
having sum(amount)>5000;
```

OUTPUT :



Result Grid | Filter Rows:

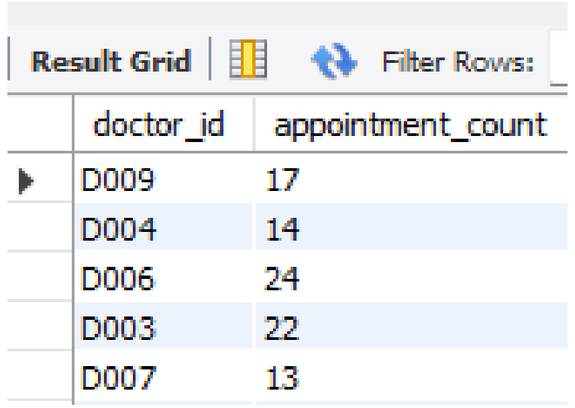
payment_status	total_amount
Pending	184612.0100000
Paid	173424.8999999
Failed	193212.94

which doctors have managed 3 or more appointments

QUERY :

```
select doctor_ id, count(appointment_ id) as
appointment_ count
from appointments
group by doctor_ id
having count(appointment_ id) >=3;
```

OUTPUT :



Result Grid | Filter Rows:

doctor_id	appointment_count
D009	17
D004	14
D006	24
D003	22
D007	13

ORDER BY CLAUSE :

Sorts the retrieved result set in ascending (ASC) or descending (DESC) order.

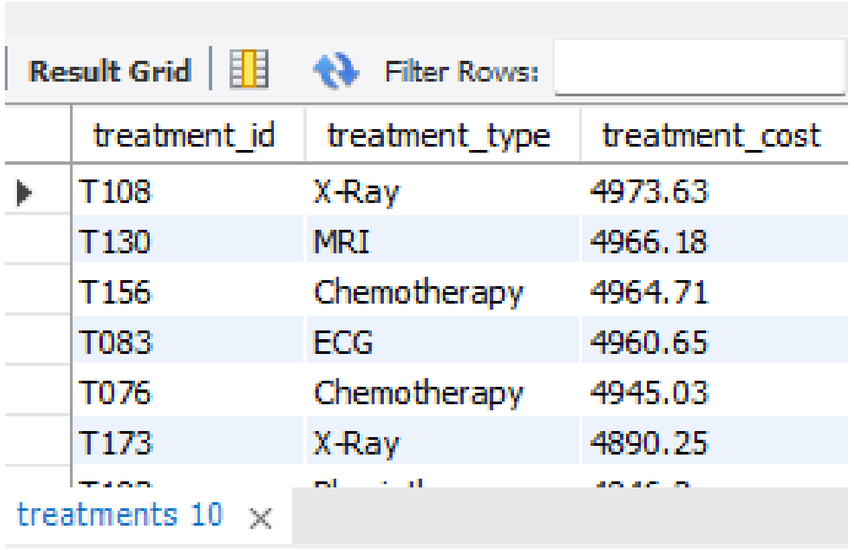
For Example :

list all treatments sorted from highest cost to lowest cost

QUERY :

```
 select treatment_id, treatment_type,
treatment_cost
from treatments
order by treatment_cost desc;
```

OUTPUT :



Result Grid | Filter Rows:

treatment_id	treatment_type	treatment_cost
T108	X-Ray	4973.63
T130	MRI	4966.18
T156	Chemotherapy	4964.71
T083	ECG	4960.65
T076	Chemotherapy	4945.03
T173	X-Ray	4890.25

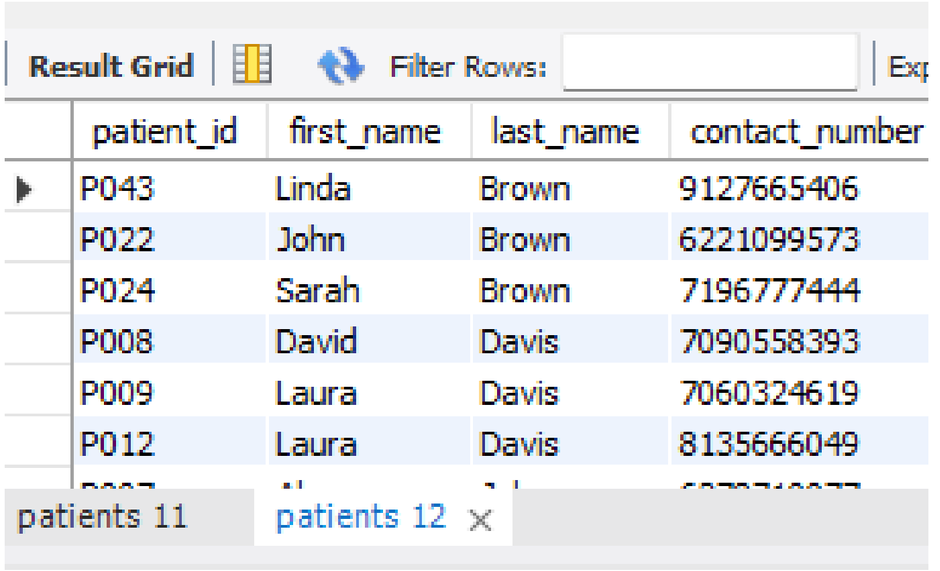
treatments 10

list all patients sorted alphabetically by their last name

QUERY :

```
select patient_id, first_name, last_name,
contact_number
from patients
order by last_name asc;
```

OUTPUT :



Result Grid | Filter Rows:

patient_id	first_name	last_name	contact_number
P043	Linda	Brown	9127665406
P022	John	Brown	6221099573
P024	Sarah	Brown	7196777444
P008	David	Davis	7090558393
P009	Laura	Davis	7060324619
P012	Laura	Davis	8135666049

patients 11 | patients 12

AGGREGATION FUNCTION :

Aggregate Functions in SQL are used to perform calculations on a set of values (a column) and return a single summary value.

SUM(): Calculates the total sum of a numeric column (e.g., total hospital revenue generated from billing).

AVG(): Computes the average value of a numeric column (e.g., average treatment cost or doctor rating).

COUNT(): Returns the total number of rows or non-null values matching a condition (e.g., total patients or appointments).

MIN(): Identifies the smallest or earliest value in a column (e.g., lowest treatment bill or first appointment date).

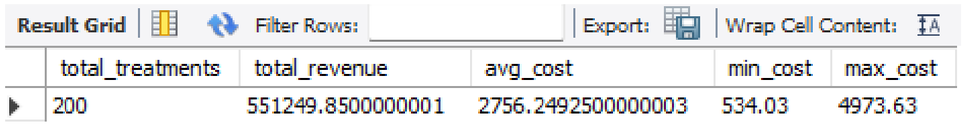
MAX(): Identifies the largest or latest value in a column (e.g., highest invoice amount or maximum doctor experience).

For Example :

QUERY :

```
select
count(*) as total_treatments,
sum(treatment_cost) as total_revenue,
avg(treatment_cost) as avg_cost,
min(treatment_cost) as min_cost,
max(treatment_cost) as max_cost
from treatments;
```

OUTPUT :



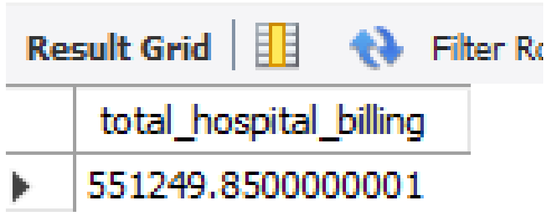
total_treatments	total_revenue	avg_cost	min_cost	max_cost
200	551249.8500000001	2756.2492500000003	534.03	4973.63

SUM :

QUERY :

```
select sum(amount) as total_hospital_billing
from billing;
```

OUTPUT :



Result Grid | Filter R

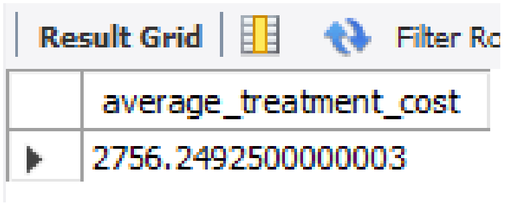
	total_hospital_billing
▶	551249.8500000001

AVG :

QUERY :

```
select avg(treatment_cost)as
average_treatment_cost
from treatments;
```

OUTPUT :



Result Grid | Filter Rc

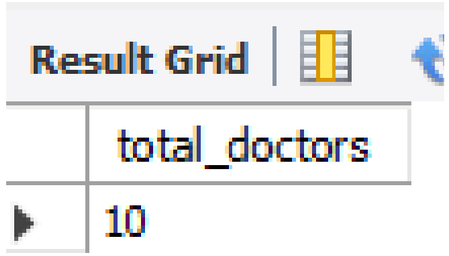
	average_treatment_cost
▶	2756.2492500000003

COUNT :

QUERY :

```
select count(doctor_id) as total_doctors
from doctors;
```

OUTPUT :



Result Grid

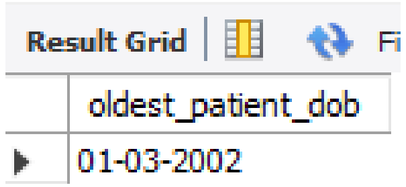
	total_doctors
▶	10

MIN :

QUERY :

```
select min(date_ of_ birth) as oldest_  patient_ dob
from patients;
```

OUTPUT :



Result Grid | Fi

	oldest_patient_dob
▶	01-03-2002

JOINS :

Joins in SQL are used to combine rows from two or more tables based on a related column between them (Primary Key / Foreign Key).

Types of Joins :

INNER JOIN : Returns only the matching records where the join key exists in both tables (e.g., listing patients who have booked appointments).

LEFT JOIN : Returns all records from the left table and the matched records from the right table (e.g., showing all doctors, even if they have no appointments).

RIGHT JOIN : Returns all records from the right table and matched records from the left table (e.g., displaying all billing transactions even if patient profile data is missing).

MULTI TABLE JOIN : Multi-Table Join is an SQL technique used to combine data from three or more tables in a single query by linking their related key columns (Primary Keys and Foreign Keys).

INNER JOIN:

show each appointment along with the patients first name,last name, and the doctors assigned to them

QUERY :

```
select
a.appointment_id,p.first_name,p.last_name,
a.doctor_id,a.appointment_date
from appointments a
inner join patients p
on a.patient_id = p.patient_id;
```

OUTPUT :

Result Grid | Filter Rows: | Export: | Wrap Cell

appointment_id	first_name	last_name	doctor_id	appointment_date
A001	Alex	Smith	D009	09-08-2023
A002	Alex	Moore	D004	09-06-2023
A003	Emily	Miller	D004	28-06-2023
A004	Robert	Wilson	D006	01-09-2023
A005	Emily	Williams	D003	06-07-2023
A006	Linda	Miller	D006	19-06-2023

Result 18

LEFT JOIN :

list all patients and their appointment details(including any patients who havent scheduled an appointment yet)

QUERY :

```
select
p.patient_id,p.first_name,p.last_name,a.appo
intment_id,a.appointment_date
from patients p
left join appointments a
on p.patient_ id = a.patient_ id;
```

OUTPUT :

Result Grid | Filter Rows: | Export: | Wrap Cell

patient_id	first_name	last_name	appointment_id	appointment_date
P001	David	Williams	A197	01-04-2023
P001	David	Williams	A071	26-01-2023
P001	David	Williams	A048	16-01-2023
P001	David	Williams	A007	09-04-2023
P002	Emily	Smith	A188	12-04-2023
P002	Emily	Smith	A082	20-01-2023

Result 19

RIGHT JOIN:

display billing details along with the patients id and billing dates

QUERY :

```
select
  p.patient_id, p.first_name, p.last_name,
 b. bill_ id, b. amount, b. payment_ status
from patients p
right join billing b
    on p. patient_ id = b. patient_ id;
```

OUTPUT :

Result Grid | Filter Rows: | Export: | Wrap Cel

patient_id	first_name	last_name	bill_id	amount	payment_status
P034	Alex	Smith	B001	3941.97	Pending
P032	Alex	Moore	B002	4158.44	Paid
P048	Emily	Miller	B003	3731.55	Paid
P025	Robert	Wilson	B004	4799.86	Failed
P040	Emily	Williams	B005	582.05	Pending
P045	Linda	Miller	B006	1381	Pending

Result 20

MULTI TABLE JOIN :

retrieve the patients full name,treatment type and treatment cost for the every recorded treatment.

QUERY :

```
select
 p. first_ name, p. lastname,t.treatment_type,
  t. treatment_ cost
from patients p
inner join appointments a
 on p.patient_id = a.patient_id
inner join treatments t
on a.appointment_id = t.appointment_id;
```

OUTPUT :

Result Grid | Filter Rows: | Export:

first_name	last_name	treatment_type	treatment_cost
Alex	Smith	Chemotherapy	3941.97
Alex	Moore	MRI	4158.44
Emily	Miller	MRI	3731.55
Robert	Wilson	MRI	4799.86
Emily	Williams	ECG	582.05
Linda	Miller	Chemotherapy	1381

Result 21

CASE WHEN :

A conditional expression that acts like an IF-THEN-ELSE statement in SQL, allowing you to return specific values based on defined conditions (e.g., categorizing patient bills as 'High' or 'Low').

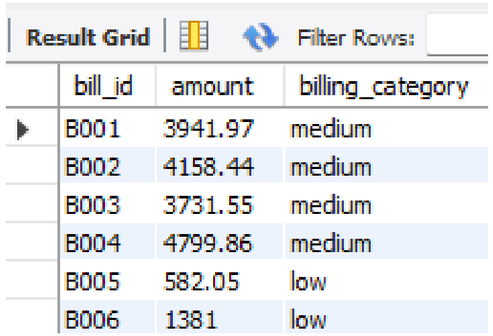
For Example :

categorize billing amounts into high, medium and low

QUERY :

```
select
bill_ id, amount,
case
when amount>=5000 then "high"
when amount between 2000 and 4999 then "medium"
else "low"
end as billing_ category
from billing;
```

OUTPUT :



Result Grid | Filter Rows:

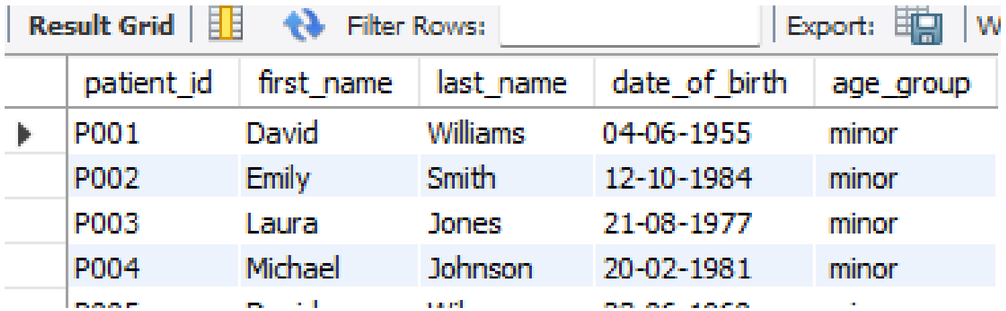
bill_id	amount	billing_category
B001	3941.97	medium
B002	4158.44	medium
B003	3731.55	medium
B004	4799.86	medium
B005	582.05	low
B006	1381	low

classify patients into senior, adult or minor based on their birth year

QUERY :

```
select
patient_ id, first_ name, last_ name, date_ of_ birth,
case
when year(date_ of_ birth)<1965 then "senior"
when year(date_ of_ birth) between 1965 and 2007 then
"adult"
else" minor"
end as age_ group
from patients;
```

OUTPUT :



Result Grid | Filter Rows: | Export: | W

patient_id	first_name	last_name	date_of_birth	age_group
P001	David	Williams	04-06-1955	minor
P002	Emily	Smith	12-10-1984	minor
P003	Laura	Jones	21-08-1977	minor
P004	Michael	Johnson	20-02-1981	minor

SUBQUERY :

A nested query written inside a main SQL statement (such as SELECT, INSERT, UPDATE, or DELETE) used to compute intermediate results for the outer query.

For Example :

find all treatments whose cost is higher than the average cost off treatments

QUERY :

```
select treatment_ id, treatment_ type, treatment_ cost
from treatments
where treatment_ cost > (select avg(treatment_ cost)
from treatments
);
```

OUTPUT :

Result Grid | Filter Rows:

treatment_id	treatment_type	treatment_cost
T001	Chemotherapy	3941.97
T002	MRI	4158.44
T003	MRI	3731.55
T004	MRI	4799.86
T008	Physiotherapy	3413.64
T009	Physiotherapy	4541.14

treatments 25

list all patients who have paid bills with an amount greater than 4000

QUERY :

```
 select patient_id,first_name,last_name
from patients
where patient_id in (select patient_id from billing
where amount>4000);
```

OUTPUT :

Result Grid | Filter Rows:

patient_id	first_name	last_name
P003	Laura	Jones
P004	Michael	Johnson
P005	David	Wilson
P009	Laura	Davis
P010	Michael	Taylor
P011	Emily	Jones

patients 26

WINDOW FUNCTIONS :

Functions that perform calculations across a set of table rows related to the current row, without collapsing the rows into a single summary output like aggregate functions do.

For Example :

assign a sequential row number to treatments ordered from highest cost to lowest

QUERY :

```
select
treatment_ id, treatment_ type, treatment_ cost,
row_ number() over (order by treatment_ cost desc) as
row_ num
from treatments;
```

OUTPUT :

Result Grid | Filter Rows: | Export:

treatment_id	treatment_type	treatment_cost	row_num
T108	X-Ray	4973.63	1
T130	MRI	4966.18	2
T156	Chemotherapy	4964.71	3
T083	ECG	4960.65	4
T076	Chemotherapy	4945.03	5
T173	X-Ray	4890.25	6

Result 27

number each patients bills sequentially orderded by bill date

QUERY :

```
 select patient_id,first_name,last_name
from patients
where patient_id in (select patient_id from billing
where amount>4000);
```

OUTPUT :

Result Grid | Filter Rows:

bill_id	patient_id	amount	patient_bill_num
B048	P001	3249.41	1
B071	P001	2960.14	2
B197	P001	975.49	3
B007	P001	534.03	4
B082	P002	3615.96	1
B188	P002	616.15	2

Result 28

RANK ():

A window function that assigns a unique rank to each row within a result set partition, leaving gaps in rank values when ties occur.

For Example :

rank all treatments by cost (highest costs get rank 1)

QUERY :

```
select
treatment_id,treatment_type,treatment_cost,
rank() over (order by treatment_cost desc)
from treatments;
```

OUTPUT :

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

treatment_id	treatment_type	treatment_cost	rank() over (order by treatment_cost desc)
T108	X-Ray	4973.63	1
T130	MRI	4966.18	2
T156	Chemotherapy	4964.71	3
T083	ECG	4960.65	4
T076	Chemotherapy	4945.03	5
T173	X-Ray	4890.25	6

Result 29

rank doctors based on the total bill amounts associated with their treatments

QUERY :

```
select bill_ id, amount,
rank() over (order by amount desc) as bill_ rank
from billing;
```

OUTPUT :

Result Grid | Filter Rows:

bill_id	amount	bill_rank
B108	4973.63	1
B130	4966.18	2
B156	4964.71	3
B083	4960.65	4
B076	4945.03	5
B173	4890.25	6

DATE FUNCTION :

Built-in SQL functions (e.g., DATEDIFF(), DATE_ADD(), YEAR()) used to manipulate, extract, and perform calculations on date and time data types.

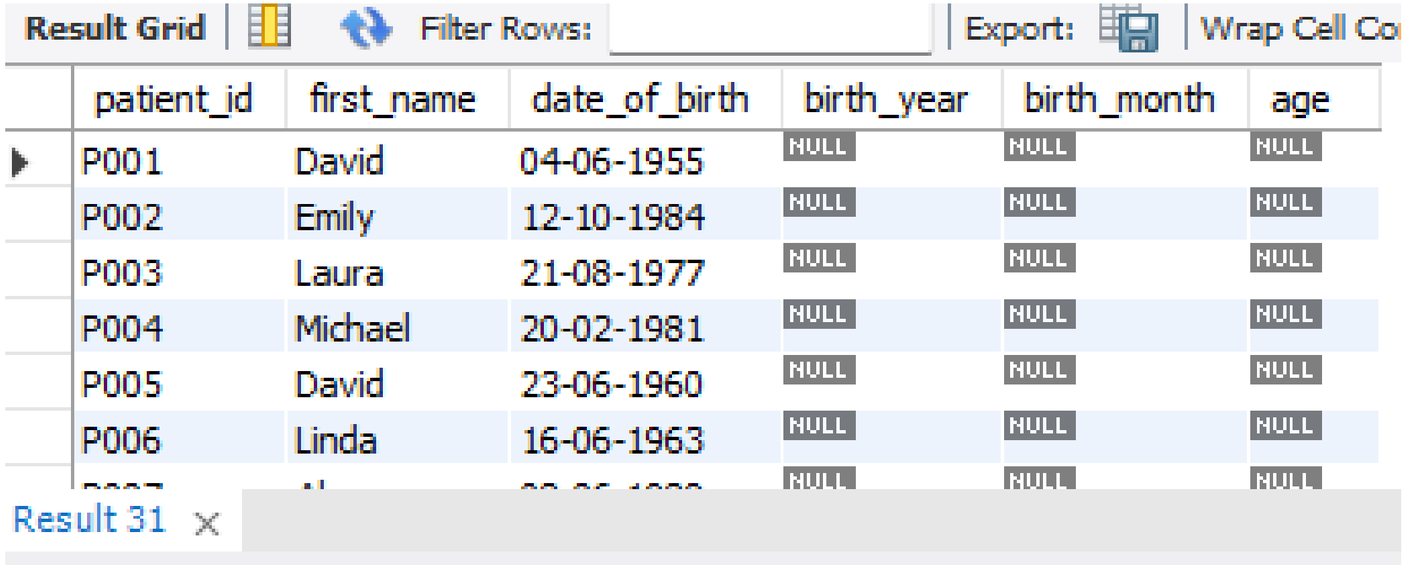
For Example :

extract the year, month name and calculate the age of each patient from their date_ of_ birth

QUERY :

```
select patient_ id, first_ name, date_of_birth,
year(date_ of_ birth) as birth_ year,
month name(date_ of_ birth) as birth_ month,
timestampdiff (year, date_ of_ birth, curdate()) as age
from patients;
```

OUTPUT :



patient_id	first_name	date_of_birth	birth_year	birth_month	age
P001	David	04-06-1955	NULL	NULL	NULL
P002	Emily	12-10-1984	NULL	NULL	NULL
P003	Laura	21-08-1977	NULL	NULL	NULL
P004	Michael	20-02-1981	NULL	NULL	NULL
P005	David	23-06-1960	NULL	NULL	NULL
P006	Linda	16-06-1963	NULL	NULL	NULL

STRING FUNCTION :

Built-in SQL functions (e.g., CONCAT(), SUBSTRING(), LENGTH(), UPPER()) used to manipulate, format, and transform text/character data.

For Example :

combine patients names into full uppercase names and extract domain names from email/contact info

QUERY :

```
select patient_ id, upper( concat (first_ name," ",last_ name)) as full_ name_ uppercase,
length(first_ name) as first_ name_ length,
substring(first_name,1,3) as name_ prefix
from patients;
```

OUTPUT :

Result Grid | Filter Rows: | Export: | Wrap

patient_id	full_name_uppercase	first_name_length	name_prefix
P001	DAVID WILLIAMS	5	Dav
P002	EMILY SMITH	5	Emi
P003	LAURA JONES	5	Lau
P004	MICHAEL JOHNSON	7	Mic
P005	DAVID WILSON	5	Dav
P006	LINDA JONES	5	Lin

Result 32

BUSINESS RECOMMENDATIONS

- **Optimize Staffing & Resources :** Adjust staff shifts and bed capacity based on peak appointment trends to reduce patient wait times.
- **Reduce Patient Readmissions :** Flag high-risk patient groups using clinical analytics to strengthen post-treatment follow-up care.
- **Accelerate Revenue Cycle :** Automate billing reminders for pending invoices to clear unpaid claims and improve cash flow.
- **Improve Data Quality :** Standardize data entry at registration and billing to eliminate incomplete patient and financial records.

CONCLUSION

This SQL project successfully transformed raw clinical, administrative, and billing data into a structured relational healthcare database. By leveraging advanced SQL techniques—including CTEs, multi-table joins, and window functions—the system delivers actionable insights into operational performance, patient care trends, and financial performance. Ultimately, the framework provides hospital administration with a scalable, data-driven foundation to optimize healthcare delivery and strategic planning.